



Arangodb une base de données multi-modèle

Projet du cours NFE204

José BERNAL

Année 2016

Table des matières

Arangodb le multi-modèle	3
Introduction	3
Présentation	4
Structure d'un cluster	5
Installation d'Arangodb	6
a) Docker	6
b) Installation standard sur Fedora 23 Gnome	7
La console de commande en ligne <code>arangosh</code>	7
Lectures/écritures	8
Réplication dans un cluster	9
Le partitionnement	10
Format d'importation Arangodb	11
Illustration des fonctionnalités avec la base discographique	12
Format des données discographiques	12
Lancement du cluster.	13
Création des collections "musics" et "labels"	14
Importation dans Arangodb	15
a) Importation en ligne de commande	15
b) Importation par l'interface web	16
Introduction au langage AQL	19
Utilisation de AQL avec la base discographique	20
FAIL OVER	25
Conclusion :	34
Références :	34
Annexes	35

Arangodb le multi-modèle

Introduction

Le choix d'un système NOSQL n'a pas été simple tant il y a pléthore de logiciels dans le monde du documentaire.

Avec les caractéristiques des différents systèmes NOSQL étudiées durant ce semestre et le site DB-Engines Ranking j'ai orienté mon choix vers Arangodb.

ArangoDB est nativement une base de données multi-modèle. On peut stocker des données paires clé/valeur, graphs ou documents et accéder aux données avec AQL, un langage de requête déclaratif commun.

Le langage de requête AQL pour Arango Query Language est dans la logique de SQL, il permet de réaliser des jointures entre les collections, des agrégations.

L'interface WEB est très complète, elle permet de réaliser la plupart des actions que ce soit la maintenance, la création de base de données et de collections, l'importation/exportation de données, le monitoring.

Des drivers sont fournis pour la plupart des langages de programmation.

Dans cette nouvelle version (3.0) la prise en charge de la recherche "fulltext" a été retravaillée, apportant la *tokenization* avec l'analyseur libicu et des indexations plus élaborées.

Après un descriptif des caractéristiques d'Arangodb j'utiliserai une base de données discographique pour illustrer la manipulation des données dans l'environnement d'Arangodb.

Par la suite je développerai ces différents aspects :

- Ses caractéristiques principales, le cluster, la scalabilité, le sharding
- Utilisation du langage déclaratif AQL, avec les jointures, le filtrage des résultats, projection des résultats, le tri, le groupement, l'agrégation.
- L'importation de données via l'interface Web et le shell Arangodb
- Le monitoring via l'interface Web
- Création / suppression de collections, document, ajout de données (JSON).
- Utilisation de l'API HTTP/REST
- Mise en place d'un cluster avec partitionnement (*shard*) et réplicas.

Présentation

ArangoDB est une base de données NoSQL développée par triAGENS GmbH, une société de conseil en informatique fondée en 2004.

La première version est sortie en 2011 sous le nom AvocadoDB, mais renommée en ArangoDB en 2012.

ArangoDB GmbH est une scission de triAGENS GmbH, société privée allemande fondée en 2014 pour répondre à la demande croissante de services professionnels autour d'ArangoDB.

Adresse postale : Hohenstaufenring 43-45 - 50674 Cologne, Germany

ArangoDB est Open-Source, publié sous licence Apache 2.0

Adresse officielle du site : <https://www.arangodb.com>

ArangoDB se veut multi-modèle, il souhaite ainsi répondre aux divers besoins documentaires, voire mixer les bases documents, Graph et clé/valeur. Le cœur d'Arangodb est écrit en C++ ce qui le rend particulièrement rapide.

Bien que jeune dans le monde NOSQL il est en constante progression, il est passé en un an de la 96^{ème} position à la 87^{ème} sur <http://db-engines.com> de l'ensemble des bases de données.

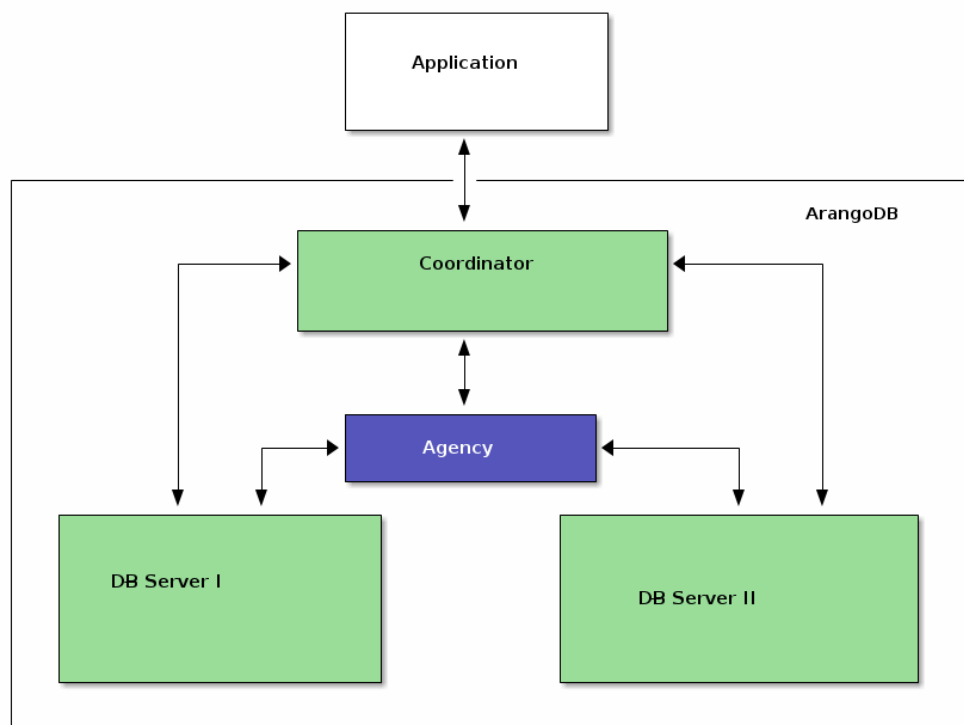
La documentation, le site en ligne, GitHub proposent tout un panel de possibilités d'installation, programmation, scripting. Des "Cookbooks" expliquent plus précisément certains aspects.

Structure Arangodb

ArangoDB peut fonctionner en autonome, en master/slave (diverses limitations) et en cluster.

En production à grande échelle c'est le mode cluster qui est choisi.

J'ai choisi le mode cluster pour le projet.



Structure d'un cluster

Un cluster ArangoDB se compose d'un certain nombre d'instances ArangoDB qui se parlent sur le réseau. Ils ont des rôles différents (exécutent le même "binaire"). La configuration actuelle du cluster est maintenue par l'*agency*, le cluster est un stockage clé/valeur très flexible et hautement disponible, basé sur un nombre impair d'instances d'*agency* qui implémentent le "**Raft Consensus Protocol**" équivalent à Paxos.

Raft Consensus Protocol :

Algorithme basé sur les *leader-follower*, le *leader* est le seul point d'entrée de toutes les opérations au sein du cluster.

Comment se déroule l'élection d'un *leader*

Suite à un time-out d'un *leader*, un replica démarrera l'élection d'un nouveau candidat *leader*.

Les replicas votent pour le candidat dont le journal est au moins aussi à jour que le leur.

Le candidat qui reçoit la majorité des voix des replicas devient le nouveau *leader*.

Le *leader* commence à envoyer des messages de "heartbeat" à tous les *followers*

Pour les différentes instances dans un cluster ArangoDB il y a 3 rôles distincts:

Agents, coordonnateurs, DBservers. Dans le cas des replicas les DBservers peuvent être *leader* ou *followers*.

Agents

Un ou plusieurs agents forment l'*agency* dans un cluster ArangoDB. L'*agency* centralise la configuration d'un cluster. Il effectue des élections de *leader* et fournit d'autres services de synchronisation pour l'ensemble du cluster. Sans *agency* les autres composants ne peuvent fonctionner.

Il est au cœur du cluster mais généralement invisible de l'extérieur. Il gère les processus de tolérance de panne.

Coordinators

Les *coordinators* sont accessibles de l'extérieur. Ils sont le point d'entrée des clients. Ils coordonnent les tâches des clusters tels que l'exécution des requêtes et des services Foxx. Ils savent où sont stockées les données et ainsi optimiser les lieux d'exécution des requêtes. Les *coordinators* sont interchangeables et peuvent être facilement arrêtés et redémarrés si nécessaire.

Primary DBservers

Les "Primary DBservers" sont les seuls à héberger des données. Ils accueillent les fragments (*shards*) et en utilisant la réplication ils peuvent être *leader* ou *follower* d'un fragment.

Ils ne doivent pas être accessibles à partir de l'extérieur, mais indirectement par l'intermédiaire des *Coordinator*". Ils peuvent également exécuter des requêtes en partie ou dans son ensemble, à la demande d'un *coordinator*.

Configuration

La configuration minimale, un *agency*, un *coordinator* et un DBServer sur une machine.

Selon l'orientation des besoins :

- Besoin de capacité et délai d'exécution des requêtes non primordiales : plus de "DBServer " que de *coordinator*
- Besoin de puissance CPU (requête/services Foxx) : plus de *coordinators* que de "DBServer"

D'autres configurations des rôles et des machines sont possibles selon le besoin.

Installation d'Arangodb

Arangodb est distribué sur la plupart des plateformes, docker, Mac, la plupart des distributions Linux, Windows, Amazon web services, et bien d'autres.

Il y a 2 packages, un serveur et un client permettant d'utiliser les outils d'Arangod à partir d'une machine n'intégrant pas le serveur.

J'ai installé Arangodb en docker et en natif sur Fedora 23 Gnome. Voici un bref descriptif des 2 méthodes.

a) Docker

L'installation d'arangodb s'effectue par le lancement d'une instance, le système télécharge une image d'arangodb s'il ne la trouve pas en local.

Voici 2 façons parmi d'autres de déclarer une instance d'arangodb dans l'environnement docker

```
docker run -e ARANGO_NO_AUTH=1 --name arangodd01 -d arangodb -v /home/jose/arangodb:/var/lib/arangodb3
```

`-e ARANGO_NO_AUTH=1` pas d'authentification sur la base arangodb (pendant les tests)

`--name arangodd01` affectation d'un nom à l'instance

Si on ne spécifie pas de nom, Arangodb en attribue un automatiquement, ce sont des personnes/événements célèbres.

`-d arangodb` exécuter le container en arrière-plan et afficher son ID

`-v /home/jose/arangodb:/var/lib/arangodb3` monter un volume du container sur un volume du host.

```
docker run -e ARANGO_NO_AUTH=1 -d arangodb -v /home/jose/arangodb:/var/lib/arangodb3
```

Il est possible de lancer Arangodb de plusieurs façons différentes avec d'autres options.

Exemple d'une installation d'une image arangodb avec instanciation, le programme est immédiatement disponible à l'usage.

```
docker run -e ARANGO_NO_AUTH=1 -d arangodb -v /home/jose/arangodb:/var/lib/arangodb3
```

```
Unable to find image 'arangodb:latest' locally
latest: Pulling from library/arangodb
5c90d4a2d1a8: Pull complete
ef41ff05e2da: Pull complete
30473b69afab: Pull complete
888d3df7196a: Pull complete
df7846a32b55: Pull complete
7eee7c22b268: Pull complete
Digest: sha256:837fdd76c9233a194d7c836c696fcadb5fa9e96f49a08d9c138671255aebbe54
Status: Downloaded newer image for arangodb:latest
2b771b4f0e5f2f72d93ba821f6fd8d2ff492cc0927b4f436a885af5c2f36103c
```

Affichage des processus des instances d'image

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2b771b4f0e5f	arangodb	"/entrypoint.sh -v /h"	4 minutes ago	Exited (1) 3 minutes ago		sleepy_franklin

L'instance peut être référencée par son nom ou par son CONTAINER ID

Démarrage d'une instance

```
docker start sleepy_franklin
```

Affichage de l'adresse IP afin d'accéder à Arangodb par le web ou le shell Arango

```
docker inspect --format '{{ .NetworkSettings.IPAddress }}' sleepy_franklin
```

```
172.17.0.2
```

Il existe 3 façons d'accéder au container Arangodb :

1. Par le web : <http://172.17.0.2:8529>
2. En installant la version client d'Arangodb
3. En accédant au shell du container hébergeant Arangodb

Voici la méthode 3, accès au *bash* du container arangodb

```
docker exec -t -i --user=root distracted_bhaskara /bin/bash
```

Mon prompt est modifié "[root@localhost jose]#" devient "root@5126159b9f6f:/#"

On est maintenant dans le shell du container Arangodb.

b) Installation standard sur Fedora 23 Gnome

Après avoir téléchargé les rpm des dernières versions :

Client: arangodb3-client-3.0.2-5.1.x86_64.rpm

Serveur : arangodb3-3.0.2-5.1.x86 64

```
dnf install arangodb3-3.0.2-5.1.x86_64
```

L'installation se déroule sans aucun problème.

Arangodb fonctionne en service Linux.

Pour des raisons de performances j'ai utilisé le cluster sur l'installation standard de Fedora 23 Gnome pour la suite de l'étude.

La console de commande en ligne arangosh.

arangosh est une console de commande en ligne qui peut être utilisé pour l'administration de ArangoDB, ainsi que l'exécution de requêtes.

```
arangosh --server.endpoint tcp://127.0.0.1:8530
```

```
arangosh (ArangoDB 3.0.1 [linux] 64bit, using VPack 0.1.30, ICU 54.1, V8 5.0.71.39, OpenSSL
1.0.1k 8 Jan 2015)
Copyright (c) ArangoDB GmbH
Pretty printing values.
Connected to ArangoDB 'http+tcp://127.0.0.1:8529' version: 3.0.1 [server], database:
'_system', username: 'root'
Type 'tutorial' for a tutorial or 'help' to see common examples
127.0.0.1:8530@ system>
```

À la suite j'utilise cette console pour exécuter la plupart des commandes d'administration.

Lectures/écritures

Ne concerne pas la réplication sur Arangodb 3.0.2

Le type d'écriture sur disque (Synchrone ou Asynchrone) dépendra du paramètre *waitForSync*

On peut modifier à tout moment ce paramètre, à la création d'une collection ou par la suite.

Par l'interface web nommée **Aardvark** ou par la commande de création en ligne (*outil arangosh*) :

```
db._create("users", {waitForSync : true}); // écriture synchrone
[ArangoCollection 105265, "users" (type document, status loaded)]
```

3 scénarios:

Asynchrone :

Les transactions sont d'abord exécutées en mémoire jusqu'à qu'à réception d'un rollback ou d'un commit.

Sur un rollback, le retour en arrière s'effectue en mémoire.

Sur un commit, les modifications sont écrites sur le disque.

La synchronisation s'effectue automatiquement vers le disque si une transaction modifie plus d'une collection.

Le reste du temps la synchronisation est asynchrone, le client recevant un acquittement alors que les données ne sont pas écrites sur disque.

Les performances sont meilleures mais la cohérence des données n'est pas garantie, un plantage pourrait altérer le système.

L'écriture asynchrone est adaptée aux lectures et aux traitements décalés.

Synchrone

Si le paramètre *waitForSync* est à *true* l'acquittement n'est donné au client que si l'écriture est effective.

Asynchrone/synchrone

Quel que soit le mode défini, il est toujours possible de faire une transaction en asynchrone ou synchrone en insérant la propriété *waitForSync* dans la transaction, la propriété ne s'appliquera qu'à cette transaction.

```
db._executeTransaction({
  collections: {
    write: "users"
  },
  waitForSync: true,
  action: function () { ... }
});
```


Réplication dans un cluster

ArangoDB organise les données des collections en "*shards*".

La réplication est synchrone en mode cluster Arangodb 3.0.2.

Elle assure la cohérence des données et la reprise automatique sur panne.

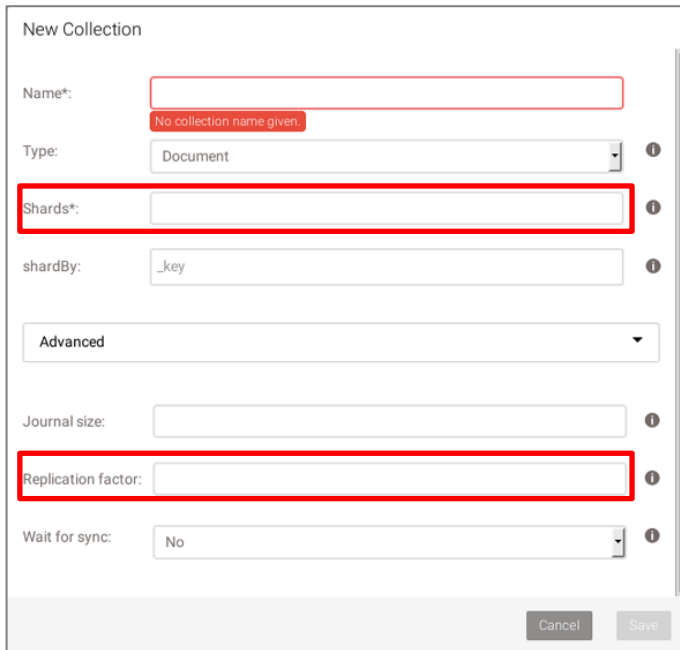
La réplication synchrone stocke une copie des données sur un autre hôte et le garde synchronisé.

On active la réplication à la création d'une collection avec le paramètre `replicationFactor > 1`

Il n'est plus modifiable par la suite.

Par le web :

Figure 1



New Collection

Name*: No collection name given.

Type:

Shards*:

shardBy:

Advanced

Journal size:

Replication factor:

Wait for sync:

Cancel Save

En commande en ligne :

```
db._create("users", { replicationFactor: 2}); // 2 rélicas  
[ArangoCollection 105511, "users" (type document, status loaded)]
```

Le *leader* (DBServerxxx) contient les fragments et le/les *followers* contiennent les répliques/backup.

Lors d'un stockage de données, le cluster attendra que toutes les répliques d'écriture soient effectuées avant de donner le feu vert au client. Cela va naturellement augmenter un peu la latence, puisqu'il faudra plus d'une étape pour chaque écriture.

Les informations sur les fragments sont partagées entre les coordonnateurs par le biais de l'*agency*.

Toutes les synchronisations des données des fragments sont effectuées de façon incrémentale, d'où des resynchronisations rapides. Cette technologie permet de déplacer des fragments (ceux de *follower* et *leader*) entre DBservers sans interruption de service.

Il est possible de transférer les fragments d'un leader ou d'un follower

Fragments	"Primaires"	Réplicas
Shard	Leader	Followers
s100071	DBServer004	DBServer001, DBServer006
s100070	DBServer003	DBServer002, DBServer005
s100069	DBServer004	DBServer001, DBServer006

Le partitionnement

Le "Sharding" permet d'utiliser plusieurs machines pour exécuter un cluster d'instances ArangoDB et constituer ainsi une seule base de données, pour des raisons de performances ou de grandes capacités de données.

Les fragments sont configurés par collection, de sorte que plusieurs fragments de données forment la collection dans son ensemble. Pour déterminer dans quel "shard" les données doivent être stockées ArangoDB effectue un hachage à travers les valeurs. Par défaut, ce hachage est créé à partir de **_key**.

Il est possible de définir une autre clé lors de la création de la collection dont la valeur devra être unique.

Le nombre de "shards" est défini à la création de la collection et ne peut être modifié par la suite.

Par la commande de création en ligne :

```
db._create("users", {numberOfShards: 2}); // création d'une collection avec 2 "shards"
[ArangoCollection 106203, "users" (type document, status loaded)]
```

Ou par l'interface web, figure 1 page précédente

Format d'importation Arangodb

- Importation JSON-encoded Data
- Importation CSV Data
- Importation TSV Data

JSON-encoded Data

Exemple :

```
{ "_key": "one", "value": 1 }  
{ "_key": "two", "value": 2 }  
{ "_key": "foo", "value": "bar" }  
...
```

Pour l'importation par lots :

```
[  
  { "_key": "one", "value": 1 },  
  { "_key": "two", "value": 2 },  
  { "_key": "foo", "value": "bar" },  
  ...  
]
```

CSV Data, une fois importées les données sont au format JSON

```
"first","last","age","active","dob"  
"John","Connor",25,true,  
"Jim","O'Brady",19,,  
"Lisa","Jones",,, "1981-04-09"  
Hans,dos Santos,0123,,  
Wayne,Brewer,,false,
```

TSV Data, la tabulation est le séparateur, il possible de spécifier un autre caractère de séparation.

Illustration des fonctionnalités avec la base discographique

Format des données discographiques

Utilisation d'une base discographique.

Site : <https://www.discogs.com/developers/>

Ce site recense des chansons, des chanteurs, des producteurs, des distributeurs etc. et donne accès à une API pour lire et proposer des contenus, les utilisateurs peuvent ajouter leurs propres informations.

J'ai créé un compte pour avoir une clé mais elle n'est pas nécessaire pour la lecture des données.

Base adresse API : <https://api.discogs.com/releases>

Artist : représente une personne dans la base Discogs : /artists/{artist_id}

Id : identification des chansons (disque, CD etc.)

Exemple de requête :

```
https://api.discogs.com/artists/1/releases?/database/search?q={nirvana}&{release}
```

J'ai choisi la méthode par l'id qui permet de balayer plus largement la base.

Ex. <https://api.discogs.com/releases/40722>

D'autres API sont disponibles (Chanteurs, producteurs, distributeurs.) :

Base adresse : <https://api.discogs.com/releases>

J'ai récupéré 2 bases :

1. la base sur la discographie, chansons (disque, CD etc.), artistes. Format JSON
2. Et sur les labels (maisons de disques). XML, je l'ai converti en JSON

Le schéma et la taille sont très variables d'un enregistrement à un autre.

J'ai mis en place un script Python qui récupère à chaque lancement 2Mo soit 200 requêtes représentant entre 0 et 200 enregistrements. Je fais des requêtes en incrémentant un **id**

Un crontab lance le script toutes les 30 minutes afin d'éviter tout risque de me faire blacklister.

Certains **id** n'existent pas, j'enregistre un fichier log au format json afin de connaître la position et les erreurs pouvant survenir lors du téléchargement. À chaque exécution du script, je lis ce fichier et le mets à jour en fin d'exécution.

Exemple fichier log

```
{
  "status": "ok",
  "url": "https://api.discogs.com/releases/68792",
  "max": 500000,
  "tempo": 2,
  "lastPage": 68792,
  "nbrMaxi": 220,
  "pasInfos": [
    1474,
    68974
  ],
  "nomFic": "20160530-140001-musics.json"
}
```

J'ai récupéré 1Go de données, ce qui est suffisant pour les tests. (70 621 docs)

L'enregistrement donne de très nombreuses informations, des liens vers Youtube, vers des pochettes images, etc.

J'ai récupéré aussi en grande partie la base "labels" :

Cela permettra de mettre en valeur la jointure, total ~860Mo (828 890 docs)

Au total près de 2Go de données.

Lancement du cluster.

Un script fournit sur gihub permet de lancer simplement les instances.

(J'ai apporté une petite modification au script afin d'avoir la persistance de mes données à chaque reboot.)

<https://github.com/arangodb/arangodb>

```
./startArangoDBClusterLocalv3.sh 1 6 2
```

Création de 1 agency, 6 DBServers et 2 coordinators

On peut aussi le faire manuellement, on commence par l'*agency*, les DBServers puis les *coordinators*.

Voici un exemple avec toutes les options (toutes ne sont pas obligatoires).

Déclaration de 1 agency

```
arangod --agency.endpoint tcp://localhost:4001 --agency.id 0 --agency.compaction-step-size 1000
--agency.election-timeout-min 0.5 --agency.election-timeout-max 2.5 --agency.notify true --
agency.size 1 --agency.supervision true --agency.wait-for-sync false --database.directory
/home/jose/arangodb/cluster/data4001 --javascript.app-path /home/jose/arangodb/cluster/jsapps --
javascript.v8-contexts 1 --log.file /home/jose/arangodb/cluster/4001.log --server.authentication
false --server.endpoint tcp://127.0.0.1:4001 --server.statistics false --server.threads 9 --
log.force-direct true
```

La déclaration des DBServers et des *coordinators* respectent une syntaxe équivalente.

On accède à l'interface Web (Aardvark) par l'un des 2 *coordinators* : <http://127.0.0.1:8530> ou 8531

L'*agency* est visible par le Web à l'adresse : <http://127.0.0.1:4001>

L'interface montre 2 coordinators et 6 DBServers

Coordinators					DB Servers				
Name	Endpoint	Heartbeat	Status	Health	Name	Endpoint	Heartbeat	Status	Health
Coordinator001	tcp://127.0.0.1:8531	01:22:27	SERVING	✓	DBServer001	tcp://127.0.0.1:8630	01:22:27	SERVING	✓
Coordinator002	tcp://127.0.0.1:8530	01:22:27	SERVING	✓	DBServer002	tcp://127.0.0.1:8629	01:22:27	SERVING	✓
					DBServer003	tcp://127.0.0.1:8631	01:22:27	SERVING	✓
					DBServer004	tcp://127.0.0.1:8633	01:22:27	SERVING	✓
					DBServer005	tcp://127.0.0.1:8632	01:22:27	SERVING	✓
					DBServer006	tcp://127.0.0.1:8634	01:22:27	SERVING	✓

Création des collections "musics" et "labels"

Le partitionnement et les replicas se définissent à la création de la collection.

Création des collections "musics" et "labels" partitionnées en 3 fragments et 3 réplicas.

Pour accéder au shell d'Arangodb (commandes en ligne)

```
arangosh --server.authentication false --server.endpoint tcp://127.0.0.1:8530
```

Commandes de création (en grisé réponse à la commande)

```
db._create("musics", {replicationFactor: 3, numberOfShards: 3})
```

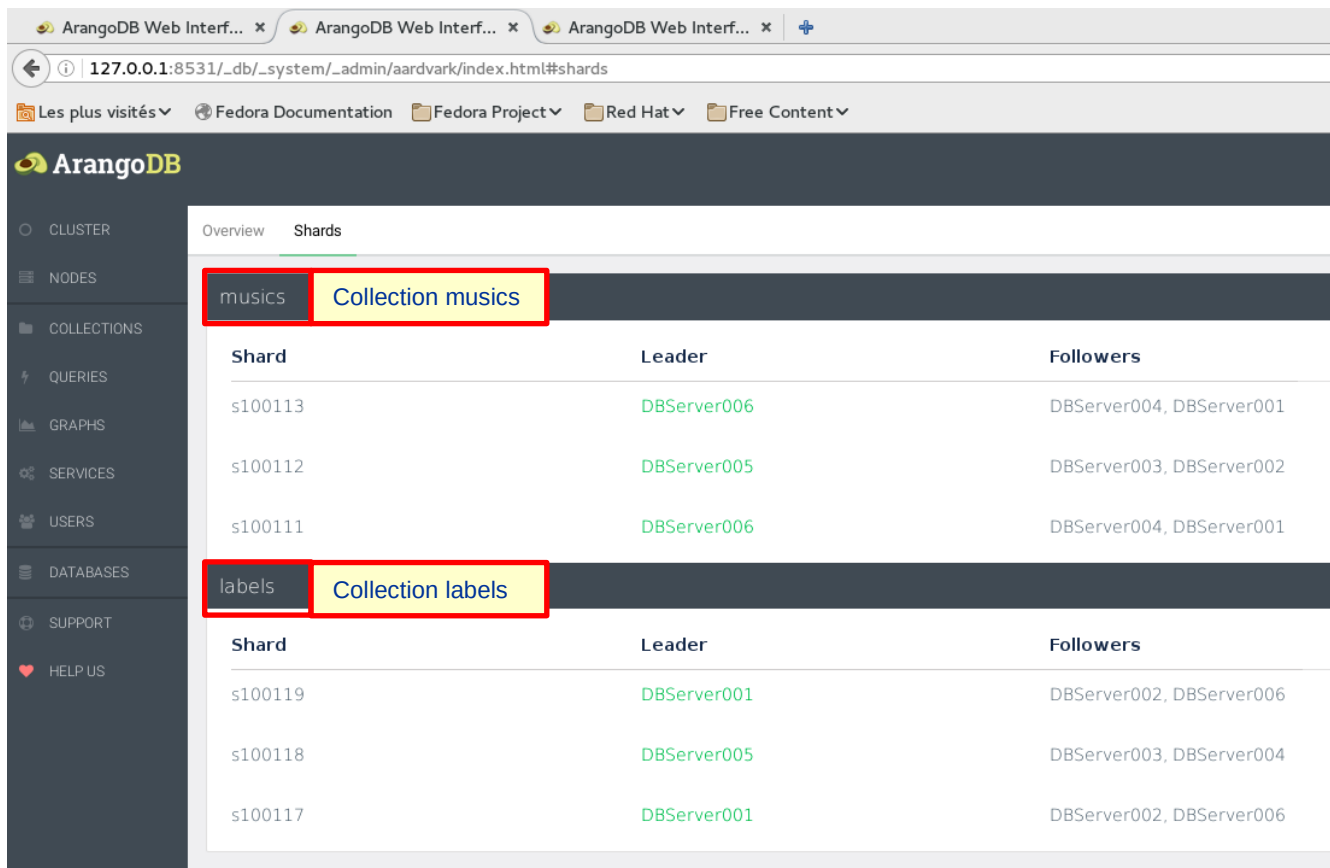
```
[ArangoCollection 100109, "musics" (type document, status loaded)]
```

```
db._create("labels", {replicationFactor: 3, numberOfShards: 3})
```

```
[ArangoCollection 100115, "labels" (type document, status loaded)]
```

Comme indiqué précédemment il est possible de créer les collections à partir de l'interface.

L'interface montre les collections "musics" et "labels" gérées par des *leaders* (avec les références des fragments) et leurs réplicas (*followers*).



The screenshot shows the ArangoDB Web Interface with the 'Shards' tab selected. The interface displays two collections: 'musics' and 'labels'. Each collection has a table of shards, their leaders, and followers.

Collection	Shard	Leader	Followers
musics	s100113	DBServer006	DBServer004, DBServer001
	s100112	DBServer005	DBServer003, DBServer002
	s100111	DBServer006	DBServer004, DBServer001
labels	s100119	DBServer001	DBServer002, DBServer006
	s100118	DBServer005	DBServer003, DBServer004
	s100117	DBServer001	DBServer002, DBServer006

Importation dans Arangodb

Il est possible d'importer des données :

1. par l'outil arangoimp
2. par l'interface web

a) Importation en ligne de commande

```
arangoimp --server.endpoint tcp://127.0.0.1:8530 --file "20160523-033001-musics.json " --  
type json --server.database "_system" --collection "musics" --create-collection true
```

```
Connected to ArangoDB 'http+tcp://127.0.0.1:8530', version 3.0.2, database: '_system',  
username: 'root'
```

```
-----  
database:           _system  
collection:         musics  
create:             yes  
source filename:    20160523-033001-musics.json  
file type:          json  
connect timeout:    5  
request timeout:    1200  
-----
```

```
Starting JSON import...
```

```
2016-07-12T22:04:16Z [5854] INFO processed 98301 bytes (3%) of input file
```

```
2016-07-12T22:04:16Z [5854] INFO processed 163835 bytes (7%) of input file
```

```
..... et ainsi de suite ....
```

```
2016-07-12T22:04:16Z [5854] INFO processed 2195389 bytes (95%) of input file
```

```
2016-07-12T22:04:16Z [5854] INFO processed 2260923 bytes (99%) of input file
```

```
created:            195
```

```
warnings/errors:    0
```

```
updated/replaced:   0
```

```
ignored:            0
```

À la fin on voit que 195 enregistrements ont été créés.

Pour importer les 500 fichiers de la base j'ai créé un script bash

```
#!/bin/bash  
for file in *.json  
do  
echo 'Traitement de $file ...'  
arangoimp --server.endpoint tcp://127.0.0.1:8530 --server.password 2010 --file "$file" --  
type json --server.database "_system" --collection "musics" --create-collection true  
done
```

--server.endpoint : adresse:port du coordinator

--file : nom du fichier

--type : json, csv, tsv

--server.database : nom de la base de données

--collection : nom de la collection

--create-collection : *true* création de la collection si celle-ci n'existe pas.

La reconnaissance des schémas est automatique, des types de données seront affectés.

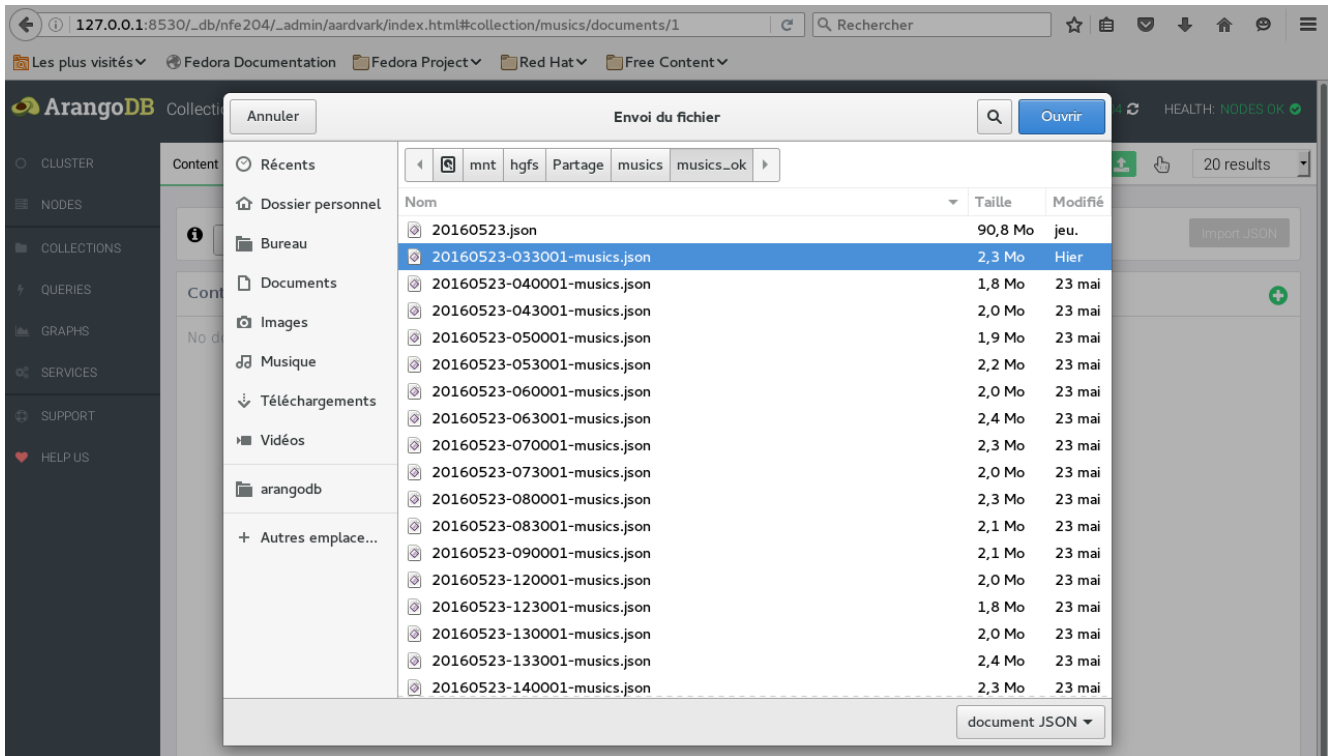
Donc, importation des fichiers dans la collection "musics" de la base de données "_system" du serveur tcp://127.0.0.1:8530

b) Importation par l'interface web

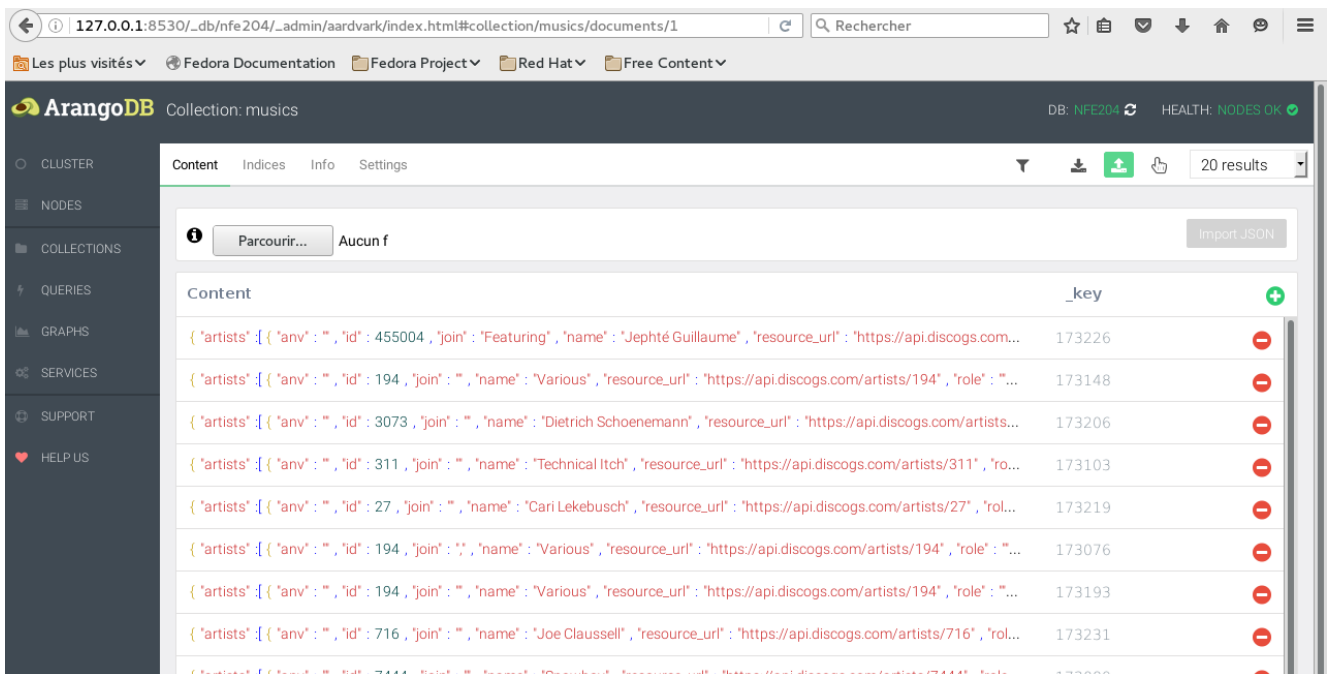
Adresse <http://127.0.0.1:8530>

Par l'intermédiaire du menu importation de la collection.

On ne peut choisir qu'un fichier à la fois et au format json.



Résultat de l'importation



Indexation

Les attributs système `_id`, `_key`, `_from` and `_to` sont automatiquement indexés par ArangoDB.

Arangodb fournit plusieurs méthodes d'indexation, en voici une, "Skip list" (listes chaînées)

Il est possible de définir un index de listes chaînées sur un ou plusieurs attributs (ou chemins) d'un document.

Création des Indexes `id` et `name` de la collection `labels` par la méthode "skiplist"

```
db.labels.ensureIndex({ type: "hash", fields: [ "id", "name" ], unique: false });
```

La commande retourne un objet (json) avec l'identifiant et les détails de l'index.

```
{
  "fields" : [
    "id",
    "name"
  ],
  "id" : "labels/169336",
  "sparse" : false,
  "type" : "hash",
  "unique" : false,
  "isNewlyCreated" : true,
  "code" : 201
}
```

Création des Indexes `id`, `name` et `labels` de la collection `musics` par la méthode "skiplist"

```
db.musics.ensureIndex({ type: "hash", fields: [ "id", "name", "labels" ], unique: false });
```

La commande retourne un objet (json) avec l'identifiant et les détails de l'index.

```
"fields" : [
  "id",
  "name",
  "labels"
],
"id" : "musics/170066",
"sparse" : false,
"type" : "hash",
"unique" : false,
"isNewlyCreated" : true,
"code" : 201
}
```

J'ai choisi ces champs en vue de la jointure.

Exemple de lecture des informations de l'index id = "labels/169336" avec curl

```
curl --dump - http://localhost:8530/_api/index/labels/169336
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Server: ArangoDB
Connection: Keep-Alive
Content-Length: 114
{
  "fields": [
    "id",
    "name"
  ],
  "id": "labels/169336",
  "sparse": false,
  "type": "hash",
  "unique": false,
  "error": false,
  "code": 200
}
```

Il est possible d'ajouter directement les champs à indexer par l'interface web.

Collection: labels DB: _SYS

Content **Indices** Info Settings

ID	Type	Unique	Sparse	Selectivity Est.	Fields	Action
0	primary	true	false	100.00%	_key	
169336	hash	false	false	n/a	id, name	 

Collection: labels

Content **Indices** Info Settings

Type: Geo Index

Fields:

Geo JSON:

Geo Index
Hash Index
Persistent Index
Fulltext Index
Skip-List Index

Collection: labels

Content **Indices** Info Settings

Type: Skip-List Index

Fields: id

Unique: ☐

Sparse: ☐

Introduction au langage AQL

Caractéristiques

Le langage de requêtes AQL (Arango Query Language) est utilisé pour lire et modifier les données stockées dans la base de données.

AQL est principalement un langage déclaratif.

AQL est indépendant du support ou du langage de programmation utilisé, sa syntaxe reste la même.

AQL supporte les modèles de requêtes complexes, puissantes et efficaces, avec les différents modèles de données, et une sémantique de transaction permettant de faire des jointures.

AQL prend en charge la lecture et la modification de données, mais pas les opérations de définition de données telles que la création et suppression des bases de données, des collections et des index.

Je retrouve dans AQL la grande partie du langage de Mongodb et de Pig Latin, avec l'agrégation, le "flatten", etc..

Il propose des requêtes pratique (via HTTP / REST et AQL), y compris des jointures entre différentes collections et graph database et la cohérence des transactions est garantie (selon configuration).

Les requêtes AQL peuvent être exécutées à partir :

- de l'interface Web,
- de l'objet db (soit en arangosh ou dans un service Foxx)
- ou par l'API HTTP.

Les requêtes sont exécutées par un coordinator, qui au besoin délègue à un DBServeur.

Dans l'interface web (Aadvark) le bouton **explain** donne des informations sur la façon dont le serveur va traiter la transaction.

The screenshot shows the ArangoDB web interface. The top navigation bar includes the ArangoDB logo and a sidebar with links to CLUSTER, NODES, COLLECTIONS, QUERIES, GRAPHS, SERVICES, USERS, DATABASES, SUPPORT, and HELP US. The main content area is titled 'Editor' and shows a query in the 'Queries' tab. The query is:

```
1 FOR doc in musics
2   FILTER doc._id == "musics/102889"
3   RETURN doc
```

The 'Explain' button is clicked, showing the execution plan and indexes used. The execution plan is as follows:

Id	NodeType	Site	Est.	Comment
1	SingletonNode	DBS	1	* ROOT
6	IndexNode	DBS	1	- FOR doc IN musics /* primary index scan */
9	RemoteNode	COOR	1	- REMOTE
10	GatherNode	COOR	1	- GATHER
5	ReturnNode	COOR	1	- RETURN doc

The indexes used are:

By	Type	Collection	Unique	Sparse	Selectivity	Fields	Ranges
6	primary	musics	true	false	100.00 %	['_key']	(doc._id == "musics/102889")

Optimization rules applied:

Id	RuleName
1	use-indexes
2	remove-filter-covered-by-index
3	scatter-in-cluster
4	remove-unnecessary-remote-scatter

Utilisation de AQL avec la base discographique

Pour ce premier exemple je montre 3 façons d'effectuer une requête.

Par la suite je n'utiliserai que l'interface web *Aardvark*.

J'affiche le document dont l'id = 720495

Depuis l'interface Web	Depuis le shell (arangosh)
<pre>FOR u IN labels FILTER u.id == 720495 RETURN u</pre>	<pre>db._query('FOR u IN labels FILTER u.id == 720495 RETURN u')</pre>
Résultat	Résultat
<pre>[{ "_id": "labels/1816617", "_key": "1816617", "_rev": "223282", "data_quality": "Needs Major Changes", "id": 720495, "name": "Salud Productions" }]</pre>	<pre>[{ "_id": "labels/1816617", "_key": "1816617", "_rev": "223282", "data_quality": "Needs Major Changes", "id": 720495, "name": "Salud Productions" }]</pre>

Avec curl :

```
curl -X POST --data-binary @- --dump - http://localhost:8530/_api/cursor <<EOF
{
  "query" : "FOR u IN labels FILTER u.id == 720495 RETURN u"
}
EOF
```

Résultat

```
{
  "result": [
    {
      "_id": "labels/1816617", "_key": "1816617", "_rev": "223282",
      "data_quality": "Needs Major Changes",
      "id": 720495,
      "name": "Salud Productions"
    }
  ]
}
```

À la suite information sur le résultat de la requête

Insertion d'un document

```
INSERT { name: "Jose", data_quality: "A tester" }
      INTO labels
```

Insertion de plusieurs documents

```
FOR name IN [
  { name: "Jose", data_quality: "A tester"},
  { name: "Jose", data_quality: "Qualite 78 tours" }
]
INSERT name INTO labels
```

Insertion d'un document d'une collection vers une autre collection

```
FOR document IN labels
  FILTER document._id == "labels/2272867"
  INSERT document INTO musics
```

Mise à jour d'un document

```
UPDATE { _key: "2272867" }
  WITH { name: "Zorro" }
  IN labels{
  Resultat
    "data_quality": "Qualite 78 tours",
    "name": "Zorro"
  }
```

Recherche de tous les documents avec country ="UK" et une limite à 10 documents.

```
FOR document IN musics
  FILTER document.country == "UK"
  LIMIT 10
  RETURN {document}
```

Recherche de tous les documents avec country ="UK" et une limite à 1 document et projection de 3 champs.

```
FOR document IN musics
  FILTER document.country == "UK"
  LIMIT 1
  RETURN {nom : document.artists[0].name, genre : document.genres, site : document.resource_url}
  Résultat :
  [
    {
      "nom": null,
      "genre": [
        "Electronic"
      ],
      "site": "https://api.discogs.com/releases/58658"
    }
  ]
```

Supprimer un champ dans un array

```
FOR doc IN musics
  FILTER doc._id == "musics/102889"
  LET alteredList = (
    FOR element IN doc.labels
      LET newItem = (UNSET(element, "resource_url2"))
      RETURN newItem)
  UPDATE doc WITH { labels: alteredList } IN musics
  OPTIONS { keepNull: false }
```

Agrégation et comptage du nombre de nom de "maisons de disques" (labels)

Recherche de tous les documents dont l'année est comprise entre 1900 et 2000, agrégation, puis limite affichage à 5, trie descendant effectué ensuite.

Sélection des champs à afficher : nom de la maison de disque (labels), nombre de différents labels dans la base

```
FOR document IN musics
FILTER (document.year > 1900 AND document.year < 2000)
  COLLECT labels = document.labels[0].name
  WITH COUNT INTO number
  LIMIT 5
  SORT labels DESC
  RETURN {labels : labels, number:number}
```

Extrait du résultat

```
[
  {
    "labels": "(:wizoo:)",
    "number": 1
  },
  {
    "labels": "\"O\" Records",
    "number": 4
  },
]
```

Un autre exemple qui rappelle MAP/REDUCE

```
FOR doc IN musics
  FILTER (doc.year > 1900 AND doc.year < 2000)
  COLLECT labels = doc.labels[0].name INTO resultat
  RETURN {"number":LENGTH(resultat), "id":resultat[*].doc._id}
```

- "resultat" est un tableau regroupant les résultats par nom de "label".
- On peut afficher n'importe quel champ comme _id par exemple.
- "number" correspond à la taille du tableau donc au nombre d'éléments regroupés.

Extrait du résultat

```
[
  {
    "number": 3,
    "id": [
      "musics/156579",
      "musics/151931",
      "musics/155246"
    ]
  },
  {
    "number": 8,
    "id": [
      "musics/155185",

```

Jointures

La collection **musics** comporte toutes les informations sur un groupe, dont l'id du "labels" (maison de disque)

La collection **labels** comporte les informations sur les maisons de disque, chaque enregistrement a un identifiant unique (**id**).

Je me propose d'effectuer une jointure entre les collections **musics** et **labels**, sur les champs **id** (**labels**) afin de sortir le nom des groupes, l'année de création, le nom du label.

Dans le chapitre des index, j'avais indexé les champs **id**, **nom** de la collection **labels** et les champs **id**, **name**, **labels** de la collection **musics** pour un traitement plus performant.

Ci-dessous **des extraits** des 2 documents qui vont servir à la jointure. Les documents complets sont en annexe.

Collection:" musics " -id: musics/133882 _rev: 56950 _key: 133882
<pre>{ "artists": [{ "anv": "", "id": 292739, "join": ",", "name": "From Within", </pre>
<pre> "labels": [{ "catno": "PLUS8036", "entity_type": "1", "entity_type_name": "Label", "id": 881911, "resource_url": "https://api.discogs.com/labels/881911" }] }, "year": 1994 }</pre>

Collection:" labels " -id: labels/1968143 _rev: 1250333 _key: 1968143
<pre>{ "data_quality": "Needs Vote", "id": 881911, "images": { "image": [</pre>
<pre>], "name": "Plus 8 Records", "parentLabel": "Plus 8 Records Ltd.", "profile": "", </pre>

Requête limité à 5 sorties et au "labels" "labels/1968143"

```
FOR groupe IN musics
  FOR mdisque IN labels
    FILTER (mdisque._id == "labels/1968143") AND
    (mdisque.id == groupe.labels[0].id)
  LIMIT 5
  RETURN {
    artiste : groupe.artists[0].name,
    annee : groupe.year,
    genres : groupe.genres,
    titre : groupe.title,
    maison_disque: mdisque.name,
    id : mdisque.id,
    label_id : mdisque.id
  }
```

Résultats, j'en affiche juste 2/5 (résultats complets en annexe.)

```
[
  {
    "artiste": "From Within",
    "annee": 1994,
    "genres": [
      "Electronic"
    ],
    "titre": "From Within I",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
  {
    "artiste": "Speedy J",
    "annee": 1997,
    "genres": [
      "Electronic"
    ],
    "titre": "Public Energy No.1",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
]
```


FAIL OVER

Mon cluster est composé de 1 *agency*, 2 *coordinators* et 6 DBServers, cela représente 9 instances du binaire Arangodb.

Les *agency* vont par nombre impair. Dans mon cas, 1 *agency* était suffisant car les tests que j'ai réalisés portent sur les *coordinators/leaders/followers*, afin de mettre en évidence la reprise sur panne et l'élection d'un nouveau *leader*.

Les *coordinators* sont totalement interchangeables ; si l'un tombe en panne, il est possible d'en utiliser un autre pour lui transmettre les requêtes, il trouvera le DBServeur contenant le fragment désiré.

J'ai 6 "machines" DBServeurs, 3 *leaders* et 3 *followers*, le partitionnement assure la performance et la réplication la sécurité.

Les collections sont fragmentées en 3 "*shards*" et il y a 3 réplicas.

Un DBServer peut être *leader* pour le "*shard*" d'une collection et *follower* pour le "*shard*" d'une autre collection. Leurs rôles s'inversent en fonction du service qu'ils rendent sur le moment.

Pour simuler les plantages des serveurs j'ai utilisé la commande suivante suivie du port correspondant à la machine.

```
curl -X DELETE http://localhost:$PORT/_admin/shutdown
```

Exemple :

```
curl -X DELETE http://localhost:8530/_admin/shutdown
```

L'interface Web des *coordinators* montrent les serveurs avec leurs noms, leurs ports, leurs "*shards*" et qui est *leader* ou follower.

Les erreurs sont visibles depuis l'interface web de l'*agency* à l'adresse :

```
http://127.0.0.1:4001/db/system/admin/aardvark/index.html#logs
```

Pour vérifier la reprise automatique sur panne, je lance 2 requêtes qui balayent complètement les 2 collections.

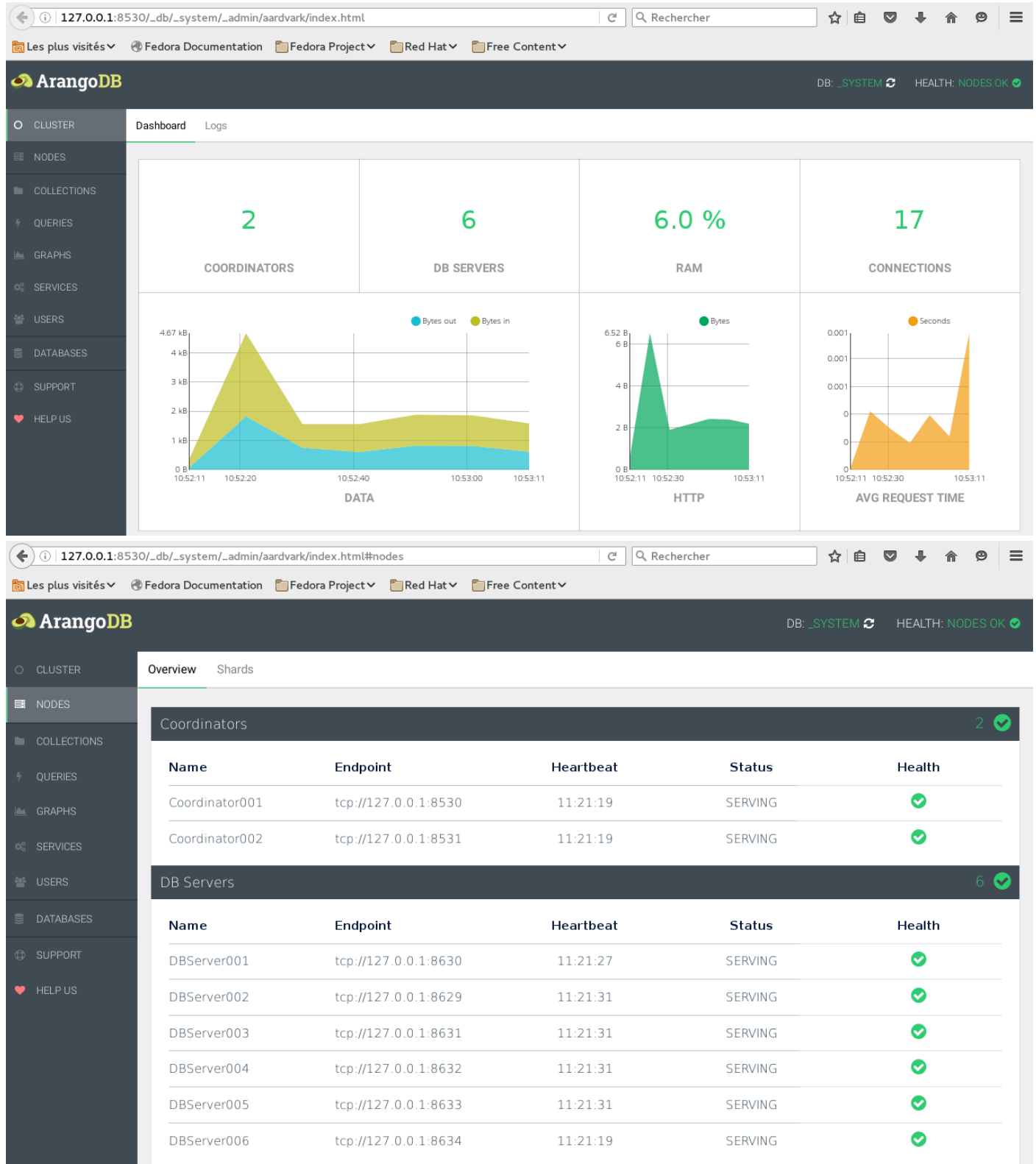
Si les 2 requêtes fonctionnent alors le système est opérationnel.

```
curl -X POST --data-binary @- --dump - http://localhost:8530/_api/cursor <<EOF
{
"query" : "FOR u IN labels RETURN u.name"
}
EOF
```

```
curl -X POST --data-binary @- --dump - http://localhost:8530/_api/cursor <<EOF
{
"query" : "FOR u IN musics RETURN u.year"
}
EOF
```

1) Début du test :

État en fonctionnement normal.



127.0.0.1:8530/_db/_system/_admin/aardvark/index.html#shards

Rechercher

Les plus visitésFedora DocumentationFedora ProjectRed HatFree Content

ArangoDB

DB: _SYSTEMHEALTH: NODES OK

CLUSTER

NODES

COLLECTIONS

QUERIES

GRAPHS

SERVICES

USERS

DATABASES

SUPPORT

HELP US

OverviewShards

musics

Shard	Leader	Followers
s100079	DBServer006	DBServer002, DBServer001
s100078	DBServer004	DBServer003, DBServer005
s100077	DBServer003	DBServer002, DBServer006

labels

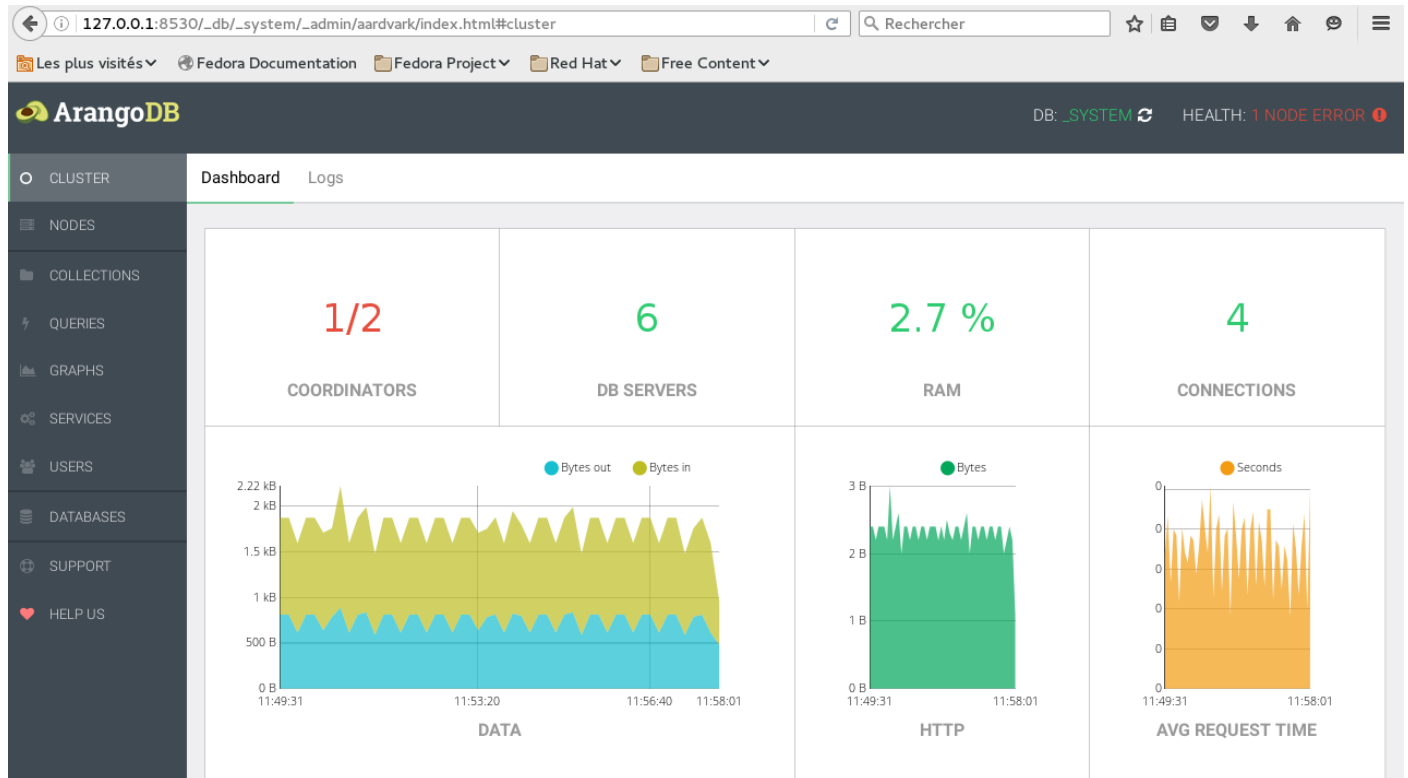
Shard	Leader	Followers
s100071	DBServer004	DBServer001, DBServer006
s100070	DBServer003	DBServer002, DBServer005
s100069	DBServer004	DBServer001, DBServer006

Rebalance Shards

2) J'arrête 1 coordinator pour simuler un plantage:

```
curl -X DELETE http://localhost:8531/_admin/shutdown
```

L'accès aux collections est toujours possible par le coordinator au port 8530



The screenshot shows the 'Overview' tab for the 'Nodes' section. It displays a table of nodes with columns: Name, Endpoint, Heartbeat, Status, and Health. The table is divided into two sections: 'Coordinators' and 'DB Servers'. The 'Coordinators' section shows two nodes: Coordinator001 (SERVING) and Coordinator002 (SHUTDOWN). The 'DB Servers' section shows six nodes, all with a status of 'SERVING'.

Name	Endpoint	Heartbeat	Status	Health
Coordinators 1 ✓				
Coordinator001	tcp://127.0.0.1:8530	11:58:09	SERVING	✓
Coordinator002	tcp://127.0.0.1:8531	11:57:54	SHUTDOWN	!
DB Servers 6 ✓				
DBServer001	tcp://127.0.0.1:8630	11:58:09	SERVING	✓
DBServer002	tcp://127.0.0.1:8629	11:58:09	SERVING	✓
DBServer003	tcp://127.0.0.1:8631	11:58:09	SERVING	✓
DBServer004	tcp://127.0.0.1:8632	11:58:09	SERVING	✓
DBServer005	tcp://127.0.0.1:8633	11:58:09	SERVING	✓
DBServer006	tcp://127.0.0.1:8634	11:58:09	SERVING	✓

3) Je stoppe DBServer006 qui est un leader.

```
curl -X DELETE http://localhost:8634/_admin/shutdown
```

The screenshot shows the ArangoDB Overview page. The top navigation bar includes the ArangoDB logo, a status bar showing 'DB: _SYSTEM' and 'HEALTH: 2 NODES ERROR', and a search bar. The left sidebar contains navigation links for CLUSTER, NODES, COLLECTIONS, QUERIES, GRAPHS, SERVICES, USERS, DATABASES, SUPPORT, and HELP US. The main content area is divided into 'Overview' and 'Shards' tabs. Under 'Overview', there are two sections: 'Coordinators' and 'DB Servers'. The 'Coordinators' section shows two entries: 'Coordinator001' (SERVING, HEALTHY) and 'Coordinator002' (SHUTDOWN, ERROR). The 'DB Servers' section shows six entries: 'DBServer001' through 'DBServer005' (all SERVING, HEALTHY) and 'DBServer006' (SHUTDOWN, ERROR). The row for 'DBServer006' is highlighted with a red border.

Name	Endpoint	Heartbeat	Status	Health
Coordinator001	tcp://127.0.0.1:8530	12:27:41	SERVING	✓
Coordinator002	tcp://127.0.0.1:8531	12:26:01	SHUTDOWN	!

Name	Endpoint	Heartbeat	Status	Health
DBServer001	tcp://127.0.0.1:8630	12:27:41	SERVING	✓
DBServer002	tcp://127.0.0.1:8629	12:27:41	SERVING	✓
DBServer003	tcp://127.0.0.1:8631	12:27:41	SERVING	✓
DBServer004	tcp://127.0.0.1:8632	12:27:41	SERVING	✓
DBServer005	tcp://127.0.0.1:8633	12:27:41	SERVING	✓
DBServer006	tcp://127.0.0.1:8634	12:26:56	SHUTDOWN	!

Puis je lance la requête qui balaye la base complète et ça fonctionne, l'accès aux données reste fonctionnel.

The screenshot shows the ArangoDB Logs page. The top navigation bar includes the ArangoDB logo, a status bar showing 'DB: _SYSTEM' and 'HEALTH: GOOD', and a search bar. The left sidebar contains navigation links for DASHBOARD, COLLECTIONS, QUERIES, GRAPHS, SERVICES, USERS, DATABASES, LOGS, SUPPORT, and HELP US. The main content area is divided into 'All', 'Info', 'Error', 'Warning', and 'Debug' tabs. The 'All' tab is selected, showing a list of log messages. The messages are filtered by 'Info' level. The messages are: 'Info 2016-07-15 12:27:16 {agency} Pending: Change leadership s100079 from DBServer006 to DBServer002', 'Info 2016-07-15 12:27:16 {agency} Todo: failed Leader for s100079 from DBServer006 to DBServer002', 'Info 2016-07-15 12:27:16 {agency} Pending: DB Server DBServer006 failed.', 'Info 2016-07-15 12:27:16 {agency} Trying to start FailedLeader job9 for server DBServer006', and 'Info 2016-07-15 12:27:16 {agency} Todo: DB Server DBServer006 failed.'. The first five messages are highlighted with a red border.

Loglevel	Date	Message
Info	2016-07-15 12:27:16	{agency} Pending: Change leadership s100079 from DBServer006 to DBServer002
Info	2016-07-15 12:27:16	{agency} Todo: failed Leader for s100079 from DBServer006 to DBServer002
Info	2016-07-15 12:27:16	{agency} Pending: DB Server DBServer006 failed.
Info	2016-07-15 12:27:16	{agency} Trying to start FailedLeader job9 for server DBServer006
Info	2016-07-15 12:27:16	{agency} Todo: DB Server DBServer006 failed.

Un nouveau *leader* a été élu, DBServer002 en remplacement de DBServer006

127.0.0.1:8530/_db/_system/_admin/aardvark/index.html#shards

Les plus visités Fedora Documentation Fedora Project Red Hat Free Content

ArangoDB DB: _SYSTEM HEALTH: 2 NODES ERROR

Overview Shards

musics

Shard	Leader	Followers
s100079	DBServer002	DBServer001, DBServer006
s100078	DBServer004	DBServer003, DBServer005
s100077	DBServer003	DBServer002, DBServer006

labels

Shard	Leader	Followers
s100071	DBServer004	DBServer001, DBServer006
s100070	DBServer003	DBServer002, DBServer005
s100069	DBServer004	DBServer001, DBServer006

Rebalance Shards

4) DBServer004 est *leader* de plusieurs fragments, je le stoppe :

```
curl -X DELETE http://localhost:8632/_admin/shutdown
```

127.0.0.1:8530/_db/_system/_admin/aardvark/index.html#nodes

Les plus visités Fedora Documentation Fedora Project Red Hat Free Content

ArangoDB DB: _SYSTEM HEALTH: 3 NODES ERROR

Overview Shards

Nodes

Coordinators 1 ✓

Name	Endpoint	Heartbeat	Status	Health
Coordinator001	tcp://127.0.0.1:8530	12:29:37	SERVING	✓
Coordinator002	tcp://127.0.0.1:8531	12:26:01	SHUTDOWN	!

DB Servers 4 ✓

Name	Endpoint	Heartbeat	Status	Health
DBServer001	tcp://127.0.0.1:8630	12:29:37	SERVING	✓
DBServer002	tcp://127.0.0.1:8629	12:29:37	SERVING	✓
DBServer003	tcp://127.0.0.1:8631	12:29:37	SERVING	✓
DBServer004	tcp://127.0.0.1:8632	12:29:27	SHUTDOWN	!
DBServer005	tcp://127.0.0.1:8633	12:29:37	SERVING	✓
DBServer006	tcp://127.0.0.1:8634	12:26:56	SHUTDOWN	!

Puis je lance la requête qui balaye la base complète et ça fonctionne, l'accès aux données reste fonctionnel.

ArangoDB Dashboard - DB: _SYSTEM HEALTH: 3 NODES ERROR

Logs

Loglevel	Date	Message
Info	2016-07-15 12:29:47	{agency} Pending: Change leadership s100078 from DBServer004 to DBServer005
Info	2016-07-15 12:29:47	{agency} Todo: failed Leader for s100078 from DBServer004 to DBServer005
Info	2016-07-15 12:29:47	{agency} Pending: Change leadership s100071 from DBServer004 to DBServer001
Info	2016-07-15 12:29:47	{agency} Todo: failed Leader for s100071 from DBServer004 to DBServer001
Info	2016-07-15 12:29:47	{agency} Pending: Change leadership s100069 from DBServer004 to DBServer001
Info	2016-07-15 12:29:47	{agency} Todo: failed Leader for s100069 from DBServer004 to DBServer001
Info	2016-07-15 12:29:47	{agency} Pending: DB Server DBServer004 failed.
Info	2016-07-15 12:29:47	{agency} Trying to start FailedLeader job10 for server DBServer004
Info	2016-07-15 12:29:47	{agency} Todo: DB Server DBServer004 failed.

Deux leaders ont été élus DBServer005 et DBServer001 en remplacement de DBServer004

ArangoDB Dashboard - DB: _SYSTEM HEALTH: 3 NODES ERROR

Shards

Shard	Leader	Followers
s100079	DBServer002	DBServer001, DBServer006
s100078	DBServer005	DBServer003, DBServer004
s100077	DBServer003	DBServer002, DBServer006

Shard	Leader	Followers
s100071	DBServer001	DBServer006, DBServer004
s100070	DBServer003	DBServer002, DBServer005
s100069	DBServer001	DBServer006, DBServer004

Rebalance Shards

5) Je stoppe DBServer001 :

```
curl -X DELETE http://localhost:8630/_admin/shutdown
```

Les requêtes ne fonctionnent plus

127.0.0.1:8530/_db/_system/_admin/aardvark/index.html#nodes

ArangoDB DB: _SYSTEM HEALTH: 4 NODES ERROR

Overview Shards

COORDINATORS

Name	Endpoint	Heartbeat	Status	Health
Coordinator001	tcp://127.0.0.1:8530	12:33:27	SERVING	✓
Coordinator002	tcp://127.0.0.1:8531	12:26:01	SHUTDOWN	!

DB Servers

Name	Endpoint	Heartbeat	Status	Health
DBServer001	tcp://127.0.0.1:8630	12:33:22	SHUTDOWN	!
DBServer002	tcp://127.0.0.1:8629	12:33:27	SERVING	✓
DBServer003	tcp://127.0.0.1:8631	12:33:27	SERVING	✓
DBServer004	tcp://127.0.0.1:8632	12:29:27	SHUTDOWN	!
DBServer005	tcp://127.0.0.1:8633	12:33:27	SERVING	✓
DBServer006	tcp://127.0.0.1:8634	12:26:56	SHUTDOWN	!

Le système n'arrive plus à se resynchroniser.

127.0.0.1:4001/_db/_system/_admin/aardvark/index.html#logs

ArangoDB DB: _SYSTEM HEALTH: 4 NODES ERROR

All Info Error Warning Debug

Loglevel	Date	Message
Error	2016-07-15 12:33:47	{agency} Failed to change leadership for shard s100071 from DBServer001 to DBServer006. No in-sync followers:["DBServer001"]
Error	2016-07-15 12:33:47	{agency} Failed to change leadership for shard s100069 from DBServer001 to DBServer006. No in-sync followers:["DBServer001"]
Error	2016-07-15 12:33:42	{agency} Failed to change leadership for shard s100071 from DBServer001 to DBServer006. No in-sync followers:["DBServer001"]
Info	2016-07-15 12:33:42	{agency} Todo: failed Leader for s100071 from DBServer001 to DBServer006
Error	2016-07-15 12:33:42	{agency} Failed to change leadership for shard s100069 from DBServer001 to DBServer006. No in-sync followers:["DBServer001"]
Info	2016-07-15 12:33:42	{agency} Todo: failed Leader for s100069 from DBServer001 to DBServer006
Info	2016-07-15 12:33:42	{agency} Pending: DB Server DBServer001 failed.
Info	2016-07-15 12:33:42	{agency} Trying to start FailedLeader job11 for server DBServer001
Info	2016-07-15 12:33:42	{agency} Todo: DB Server DBServer001 failed.

En redémarrant les serveurs, le cluster redevient opérationnel.

Élection d'un leader

Si le *leader* n'arrive pas à synchroniser les écritures avec le *follower*, au bout de 3s en échecs, il continue le service sans le *follower*, dès que celui-ci redevient disponible il synchronise les données.

Si un *leader* ne devient plus accessible (15s), l'*agency* chargé de surveiller en permanence l'ensemble des nœuds du système par envoi périodique de messages de contrôle (heartbeat) va promouvoir un autre *leader* qui contient les "shards" demandés. Si l'ancien *leader* revient, il synchronisera ses données et il deviendra un *follower*.

Conclusion :

Arangodb est un produit jeune mais qui a su tirer profit de l'expérience des autres systèmes Nosql.

Les équipes de développeurs sont très dynamiques et à chaque version d'impressionnantes nouveautés sont ajoutées.

Le langage AQL est très puissant, il rassemble les avantages de Mongodb, Ping Latin et bien d'autres.

AQL est simple dans son langage mais riche de nombreuses fonctions, l'interface graphique est très pratique pour la création des requêtes complexes.

Le langage Javascript Foxx permet l'intégration de mini services tournant dans le serveur et accessible avec AQL.

Il est d'ailleurs possible de charger des "applis" depuis le menu services comme sur "Apple Store" et "Androïd"

L'interface Web graphique Aardvark est très puissante et riche en fonctionnalités, le contenu de son affichage est adapté au serveur sur lequel elle est connectée, *agency*, DBServer, coordinator, etc.

Les prochaines versions laissent augurer de nombreuses améliorations.

Avant ce cours je ne connaissais pas du tout les coulisses du Nosql. Le cours nfe204 m'a permis de mieux aborder ce logiciel, de préparer correctement les requêtes, les tests de failover et bien d'autres choses.

Ce cours et ce projet m'ont demandé un investissement très important, mais dans ces dernières lignes je suis satisfait du résultat et des connaissances acquises.

Références :

- Adresse officiel du site : <https://www.arangodb.com>
- Arangodb GitHub : <https://github.com/arangodb/arangodb>
- NFE204 : Bases documentaires et NoSQL : <http://b3d.bdpedia.fr/>
 - Professeur Philippe Rigaux (CNAM PARIS) : <http://www.bdpedia.fr/>
- Discogs : <https://www.discogs.com>
- Et Wikipedia

Annexes

Enregistrement complet d'un document de la collection "labels"

[id:labels/1968143](#)

[rev:1250333](#)

[key:1968143](#)

```
{
  "data_quality": "Needs Vote",
  "id": 881911,
  "images": {
    "image": [
      {
        "@height": 600,
        "@type": "primary",
        "@uri": "",
        "@uri150": "",
        "@width": 600
      },
      {
        "@height": 225,
        "@type": "secondary",
        "@uri": "",
        "@uri150": "",
        "@width": 225
      },
      {
        "@height": 111,
        "@type": "secondary",
        "@uri": "",
        "@uri150": "",
        "@width": 165
      },
      {
        "@height": 250,
        "@type": "secondary",
        "@uri": "",
        "@uri150": "",
        "@width": 250
      },
      {
        "@height": 262,
        "@type": "secondary",
        "@uri": "",
        "@uri150": "",
        "@width": 271
      }
    ]
  },
  "name": "Plus 8 Records",
  "parentLabel": "Plus 8 Records Ltd.",
  "profile": "",
  "urls": {
    "url": [
      "http://www.plus8.com",
      "http://plus8records.bandcamp.com",
      "http://www.facebook.com/Plus8Records",
      "http://www.myspace.com/plus8rec",
      "http://twitter.com/plus8records",
      "http://www.youtube.com/Plus8Official",
      "http://en.wikipedia.org/wiki/Plus_8"
    ]
  }
}
```

Enregistrement complet d'un document de la collection "musics"

[id:musics/133882](#)

[rev:56950](#)

[key:133882](#)

```
{
  "artists": [
    {
      "anv": "",
      "id": 292739,
      "join": ",",
      "name": "From Within",
      "resource_url": "https://api.discogs.com/artists/292739",
      "role": "",
      "tracks": ""
    }
  ],
  "community": {
    "contributors": [
      {
        "resource_url": "https://api.discogs.com/users/mts02",
        "username": "mts02"
      },
      {
        "resource_url": "https://api.discogs.com/users/cthulhu303",
        "username": "cthulhu303"
      },
      {
        "resource_url": "https://api.discogs.com/users/SoulDancer",
        "username": "SoulDancer"
      },
      {
        "resource_url": "https://api.discogs.com/users/mayday",
        "username": "mayday"
      },
      {
        "resource_url": "https://api.discogs.com/users/elexxii",
        "username": "elexxii"
      },
      {
        "resource_url": "https://api.discogs.com/users/Bong",
        "username": "Bong"
      },
      {
        "resource_url": "https://api.discogs.com/users/DetroitBootyBass",
        "username": "DetroitBootyBass"
      },
      {
        "resource_url": "https://api.discogs.com/users/Mr.Slut",
        "username": "Mr.Slut"
      },
      {
        "resource_url": "https://api.discogs.com/users/CarlJ",
        "username": "CarlJ"
      }
    ],
    "data_quality": "Correct",
    "have": 234,
    "rating": {
      "average": 4.09,
      "count": 43
    },
    "status": "Accepted",
    "submitter": {
      "resource_url": "https://api.discogs.com/users/mts02",
      "username": "mts02"
    },
    "want": 101
  }
}
```

```

},
"companies": [
  {
    "catno": "",
    "entity_type": "9",
    "entity_type_name": "Distributed By",
    "id": 47,
    "name": "IntelliNET",
    "resource_url": "https://api.discogs.com/labels/47"
  },
  {
    "catno": "",
    "entity_type": "14",
    "entity_type_name": "Copyright (c)",
    "id": 881911,
    "name": "Plus 8 Records",
    "resource_url": "https://api.discogs.com/labels/881911"
  }
],
"country": "Canada",
"data_quality": "Correct",
"date_added": "2002-06-09T23:27:42-07:00",
"date_changed": "2015-08-31T12:22:36-07:00",
"estimated_weight": 230,
"extraartists": [
  {
    "anv": "",
    "id": 1910,
    "join": "",
    "name": "Pete Namlook",
    "resource_url": "https://api.discogs.com/artists/1910",
    "role": "Producer, Written-By",
    "tracks": ""
  },
  {
    "anv": "",
    "id": 835,
    "join": "",
    "name": "Richie Hawtin",
    "resource_url": "https://api.discogs.com/artists/835",
    "role": "Producer, Written-By",
    "tracks": ""
  }
],
"format_quantity": 1,
"formats": [
  {
    "descriptions": [
      "12\"",
      "33 ? RPM"
    ],
    "name": "Vinyl",
    "qty": "1"
  }
],
"genres": [
  "Electronic"
],
"id": 34570,
"identifiers": [],
"images": [
  {
    "height": 307,
    "resource_url": "",
    "type": "primary",
    "uri": "",
    "uri150": "",
    "width": 307
  }
]

```

```

    },
    {
      "height": 307,
      "resource_url": "",
      "type": "secondary",
      "uri": "",
      "uri150": "",
      "width": 307
    }
  ],
  "labels": [
    {
      "catno": "PLUS8036",
      "entity_type": "1",
      "entity_type_name": "Label",
      "id": 881911,
      "resource_url": "https://api.discogs.com/labels/881911"
    }
  ],
  "notes": "Contains two tracks from the full-length album originally released on FAX, Germany. CD re-released in 2000 on Minus as FWL/4.\r\nThe track \"A Million Miles To Earth\" have the last 7 minutes removed compared to the CD version.\r\n",
  "released": "1994",
  "released_formatted": "1994",
  "resource_url": "https://api.discogs.com/releases/34570",
  "series": [],
  "status": "Accepted",
  "styles": [
    "Ambient"
  ],
  "thumb": "",
  "title": "From Within I",
  "tracklist": [
    {
      "duration": "22:17",
      "position": "A",
      "title": "A Million Miles To Earth",
      "type_": "track"
    },
    {
      "duration": "12:20",
      "position": "AA",
      "title": "Homeward Bound",
      "type_": "track"
    }
  ],
  "uri": "https://www.discogs.com/From-Within-From-Within-I/release/34570",
  "videos": [
    {
      "description": "Pete Namlook & Richie Hawtin - A million miles to earth",
      "duration": 602,
      "embed": true,
      "title": "Pete Namlook & Richie Hawtin - A million miles to earth",
      "uri": "https://www.youtube.com/watch?v=am8UbUMxNoY"
    },
    {
      "description": "Pete Namlook & Richie Hawtin - Homeward Bound",
      "duration": 741,
      "embed": true,
      "title": "Pete Namlook & Richie Hawtin - Homeward Bound",
      "uri": "https://www.youtube.com/watch?v=Cgd7ZWCgtAo"
    }
  ],
  "year": 1994
}

```

Résultat de la jointure entre la collection "musics" et "labels"

```
[
  {
    "artiste": "From Within",
    "annee": 1994,
    "genres": [
      "Electronic"
    ],
    "titre": "From Within I",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
  {
    "artiste": "Speedy J",
    "annee": 1997,
    "genres": [
      "Electronic"
    ],
    "titre": "Public Energy No.1",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
  {
    "artiste": "Speedy J",
    "annee": 1991,
    "genres": [
      "Electronic"
    ],
    "titre": "Evolution E.P.",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
  {
    "artiste": "Various",
    "annee": 1991,
    "genres": [
      "Electronic"
    ],
    "titre": "From Our Minds To Yours Vol. 1",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  },
  {
    "artiste": "Cyberpersonik",
    "annee": 1991,
    "genres": [
      "Electronic"
    ],
    "titre": "Backlash",
    "maison_disque": "Plus 8 Records",
    "id": 881911,
    "label_id": 881911
  }
]
```